IT4931 / IT5384 / IT4043E · SOICT · HUST

Spark ML

Lecture 13 — DataFrame ML · Pipelines · tuning

spark.ml · Transformer · Estimator · CrossValidator

Learning outcomes (CLO2, CLO3)

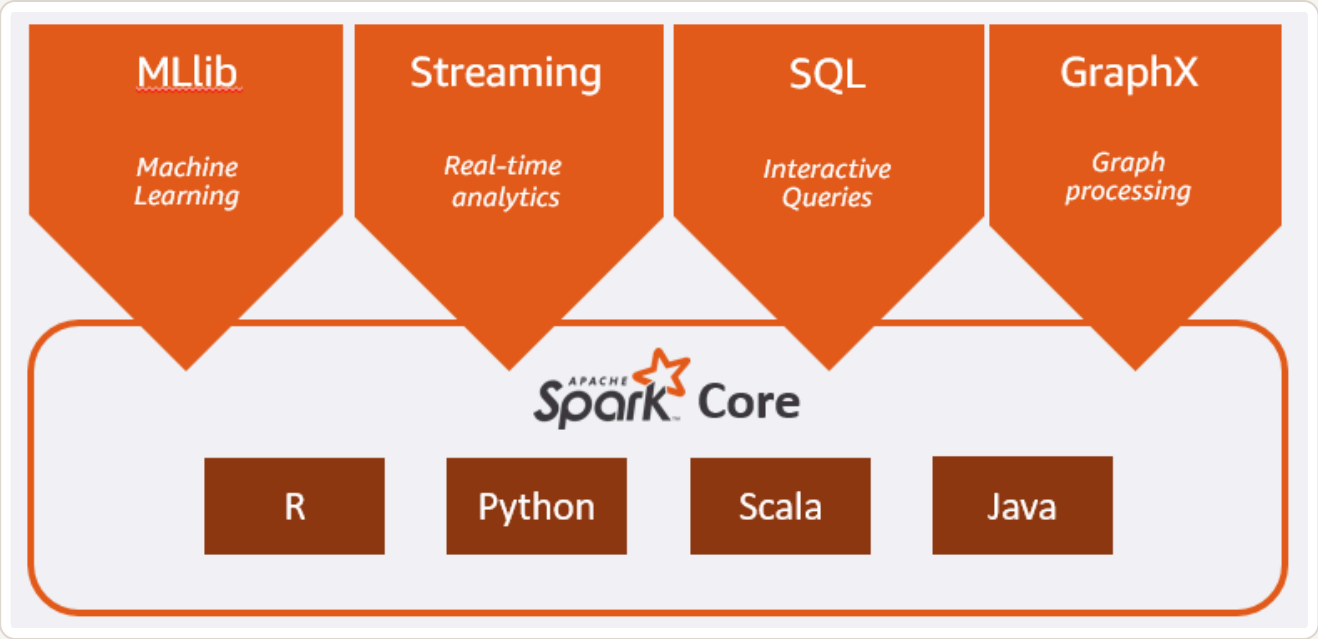
1. **Contrast** `spark.mllib` (RDD, legacy) vs `spark.ml` (DataFrame)
2. **Use** Transformer / Estimator / Pipeline / PipelineModel correctly
3. **Assemble** a feature → model chain and evaluate predictions
4. **Tune** with `ParamGridBuilder` + `CrossValidator` / `TrainValidationSplit`
5. **Outline** an end-to-end lab demo (load → pipeline → metrics → save)

Roadmap

1. **13.1** Concepts — DataFrame ML, Transformer, Estimator
2. **13.2** Pipelines & PipelineModel
3. **13.3** Features · algorithms · evaluation · tuning
4. **13.4** Demo outline · takeaways

13.1 Concepts

Where ML sits in the Spark stack



Same engine as batch (L10) and streaming (L11) — train & score on lake-scale **DataFrames**.

What MLlib provides

From the MLlib Guide:

Tooling	Examples
ML Algorithms	Classification, regression, clustering, collaborative filtering
Featurization	Extract · transform · reduce · select
Pipelines	Build · evaluate · **tune** workflows
Persistence	Save / load algorithms, models, Pipelines
Utilities	Linear algebra, statistics, data handling

Goal: practical ML that is **scalable** and **easy** on Spark.

Two packages — prefer DataFrame API

	`spark.mllib`	`spark.ml`
Data	RDD	**DataFrame** (Spark SQL)
Role	RDD-based API	DataFrame Pipelines API
Status	**Maintenance mode** (bugfixes only)	**Primary** API since Spark 2.0

Official note: MLlib is **not deprecated** — it includes both APIs. “Spark ML” is informal slang for `spark.ml`.

Course focus: `pyspark.ml` / `org.apache.spark.ml`.

DataFrame as the ML dataset

Official ML Pipelines guide: a DataFrame can hold mixed columns — text, vectors, labels, predictions.

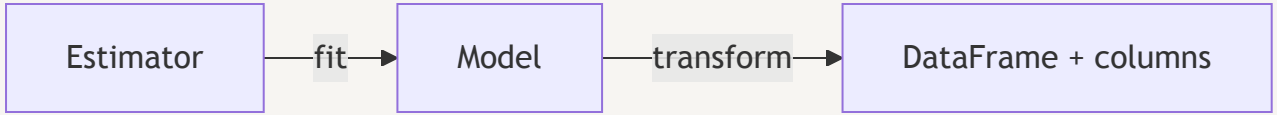
Typical column	Role
`features`	`Vector` of model inputs (`VectorAssembler` , TF, …)
`label`	Supervised target
`prediction` / `probability`	Model outputs after `transform`
raw / id / text	Domain fields before featurization

Everything is **column-oriented** — same mental model as Spark SQL (L10).

Transformer vs Estimator

Per ML Pipelines (scikit-learn–inspired):

	Transformer	Estimator
Call	`transform(df)` → new DF	`fit(df)` → **Model** (a Transformer)
Effect	Usually **appends** column(s)	Learns parameters from data
Examples	`Tokenizer` , `HashingTF` , **trained** `*Model`	`LogisticRegression` , `StandardScaler` , `Pipeline`



`fit` / `transform` are **stateless** APIs; each stage instance has a unique ID (for Params).

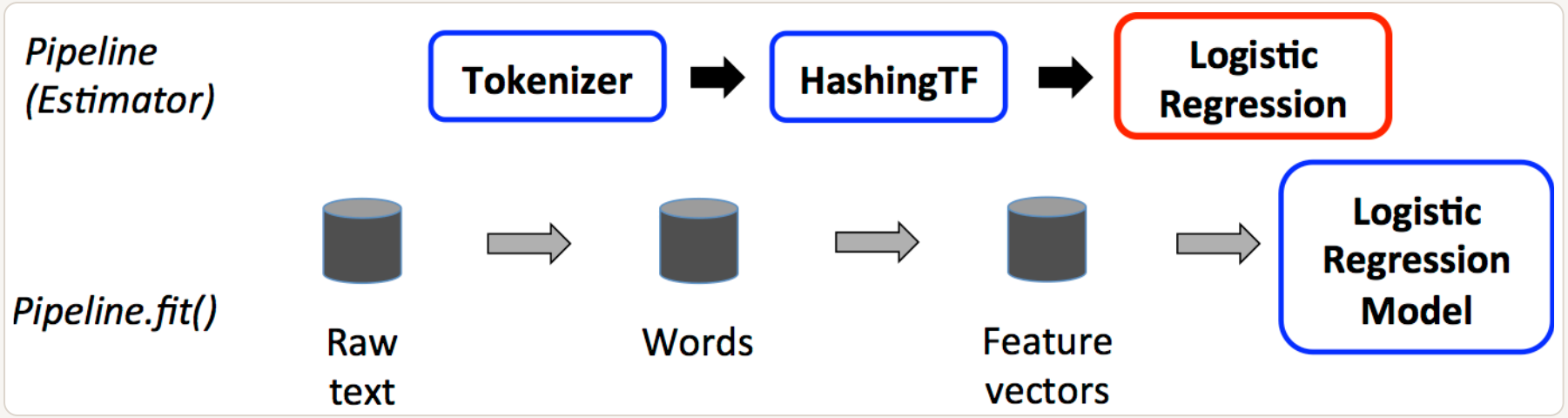
Parameters (shared API)

- Every stage exposes **Params** (`inputCol` , `outputCol` , `regParam` , …)
- Set via setters or a **ParamMap** at `fit` time
- Important for tuning: grids search over Params of **any** stage in a Pipeline

```
lr = LogisticRegression(maxIter=10, regParam=0.01)
print(lr.explainParams())          # inspect
model = lr.fit(train)              # uses default / set params
model = lr.fit(train, {lr.regParam: 0.1, lr.maxIter: 20})
```

13.2 Pipelines

Pipeline = Estimator of stages



Pipeline = ordered stages (Transformers + Estimators). `fit` trains Estimators in order → **PipelineModel**.

What `Pipeline.fit` does

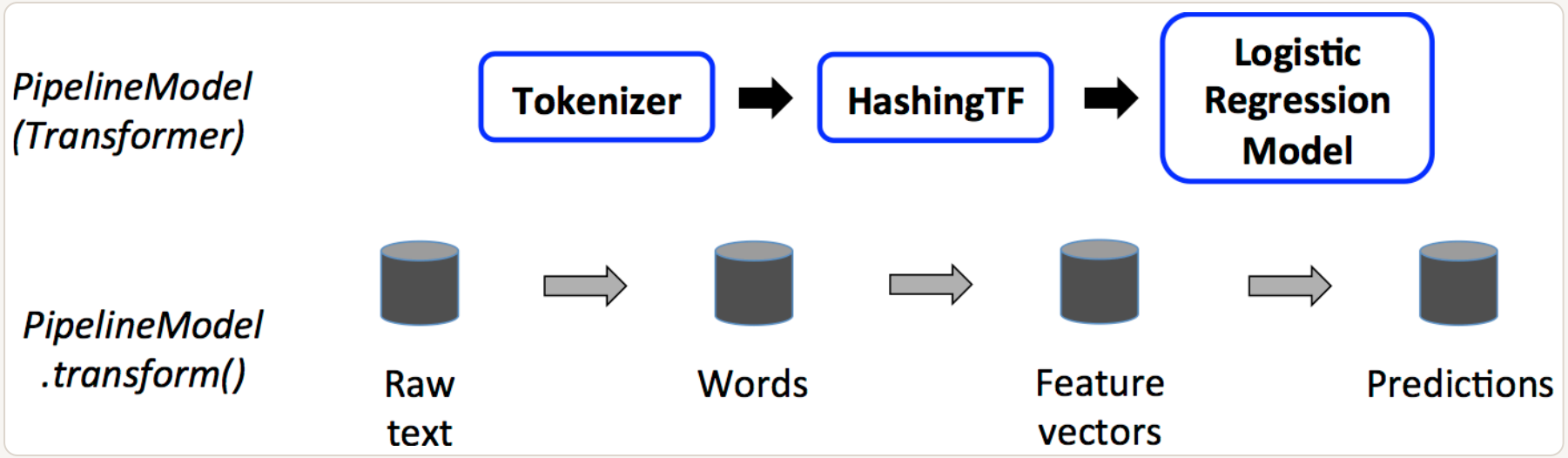


Per stage, in order (docs):

1. **Transformer** → `transform` (usually append columns)
2. **Estimator** → `fit` → **Model**, then that Model's `transform`
3. Result: **PipelineModel** (same #stages, all Transformers)

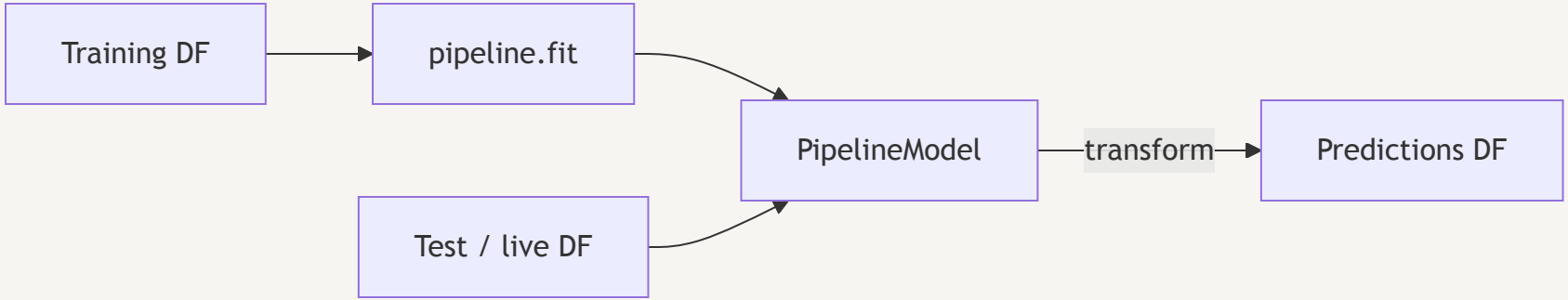
Train and test then share identical feature steps.

PipelineModel after fit



All stages are Transformers now · `transform(test)` applies the **same** feature path · then save/load for serving.

Fit / transform flow



1. Build stages (featurize → estimate)
2. `fit` on **train only**
3. `transform` holdout / stream batch
4. Evaluate · persist model

Code — text classification Pipeline

```python
from pyspark.ml import Pipeline
from pyspark.ml.feature import Tokenizer, HashingTF
from pyspark.ml.classification import LogisticRegression

tokenizer = Tokenizer(inputCol="text", outputCol="words")
hashingTF = HashingTF(inputCol="words", outputCol="features")
lr = LogisticRegression(maxIter=10)

pipe = Pipeline(stages=[tokenizer, hashingTF, lr])
model = pipe.fit(train_df)                  # PipelineModel
preds = model.transform(test_df)            # + prediction, probability, …
```

Classic example from the ML Pipelines guide.

Numeric features variant

```
from pyspark.ml.feature import VectorAssembler, StandardScaler
from pyspark.ml.classification import LogisticRegression
from pyspark.ml import Pipeline

assembler = VectorAssembler(inputCols=["age", "income", "score"],
                            outputCol="features")
scaler = StandardScaler(inputCol="features", outputCol="scaled",
                        withStd=True, withMean=False)
lr = LogisticRegression(featuresCol="scaled", labelCol="label")

pipe = Pipeline(stages=[assembler, scaler, lr])
```

Leakage rule: fit `StandardScaler` (and encoders) **inside** the Pipeline on train folds only.

13.3 Features · algorithms · eval · tuning

Featurization (high level)

Group	Examples	Use
Text	`Tokenizer` , `HashingTF` , `IDF` , `Word2Vec`	Bags of words / embeddings
Assemble	`VectorAssembler`	Merge columns → `features`
Scale / norm	`StandardScaler` , `MinMaxScaler` , `Normalizer`	Numeric prep
Encode	`StringIndexer` , `OneHotEncoder`	Categorical
Reduce	`PCA` , `ChiSqSelector`	Dimensionality / selection

All are Pipeline stages (`pyspark.ml.feature`).

Algorithm families

Family	Examples	Task
Classification	LogisticRegression, RF, GBT, LinearSVC	Labels
Regression	LinearRegression, GBTRegressor, …	Continuous (y)
Clustering	KMeans, BisectingKMeans, GaussianMixture	Unsupervised
Recommenders	ALS	Collaborative filtering
Other	FP-Growth, IsotonicRegression, …	Mining / specialty

Same pattern: Estimator → Model → `transform`.

Evaluation

Task	Evaluator / metrics
Binary class	`BinaryClassificationEvaluator` — areaUnderROC, areaUnderPR
Multiclass	`MulticlassClassificationEvaluator` — accuracy, f1, …
Regression	`RegressionEvaluator` — RMSE, MAE, (R^2)
Ranking / clustering	specialized evaluators in `pyspark.ml.evaluation`

Always pass the **prediction** (or rawPrediction) column the model wrote.

Model selection — CrossValidator

From ML Tuning — needs three things:

1. **Estimator** (often a whole `Pipeline`)
2. **ParamMaps** (`ParamGridBuilder`)
3. **Evaluator** (ROC-AUC, RMSE, …)

`CrossValidator` (e.g. (k=3)): each fold trains on (2/3), tests on (1/3); averages the metric.
Then **re-fits** the best `ParamMap` on the **entire** dataset.

Cost example: (3 \times 2) grid × (2) folds = **12** models. Prefer (k \ge 3) in practice.

```
cv = CrossValidator(estimator=pipeline,
                    estimatorParamMaps=paramGrid,
                    evaluator=BinaryClassificationEvaluator(),
                    numFolds=3)
cvModel = cv.fit(training)
```

TrainValidationSplit (cheaper alternative)

Same ingredients as CV, but **one** train/validation split (ratio configurable) — fewer model fits.

Tool	When
`CrossValidator`	More statistically sound; expensive
`TrainValidationSplit`	Faster exploration / large grids

Both can evaluate ParamMaps in **parallel** (`parallelism` ≥ 2) — pick carefully vs cluster size.

Persistence — save / load Pipelines

First-class in the MLlib guide: save algorithms, **models**, and **Pipelines**.

```
model.write().overwrite().save("/models/spam-pipeline")
from pyspark.ml import PipelineModel
loaded = PipelineModel.load("/models/spam-pipeline")
loaded.transform(new_df)
```

Use the **same PipelineModel** in batch jobs or structured streaming scoring — identical feature stages.

Leakage & good practice

- **Fit** scalers / indexers / IDF **only on train** (keep them *inside* the Pipeline so CV folds stay clean)
- Do **not** fit on full data then split
- Prefer tuning the **whole Pipeline**, not stages by hand (tuning guide)
- Persist PipelineModel; track metrics + Params (lab → later: registry / MLflow)

13.4 Demo outline

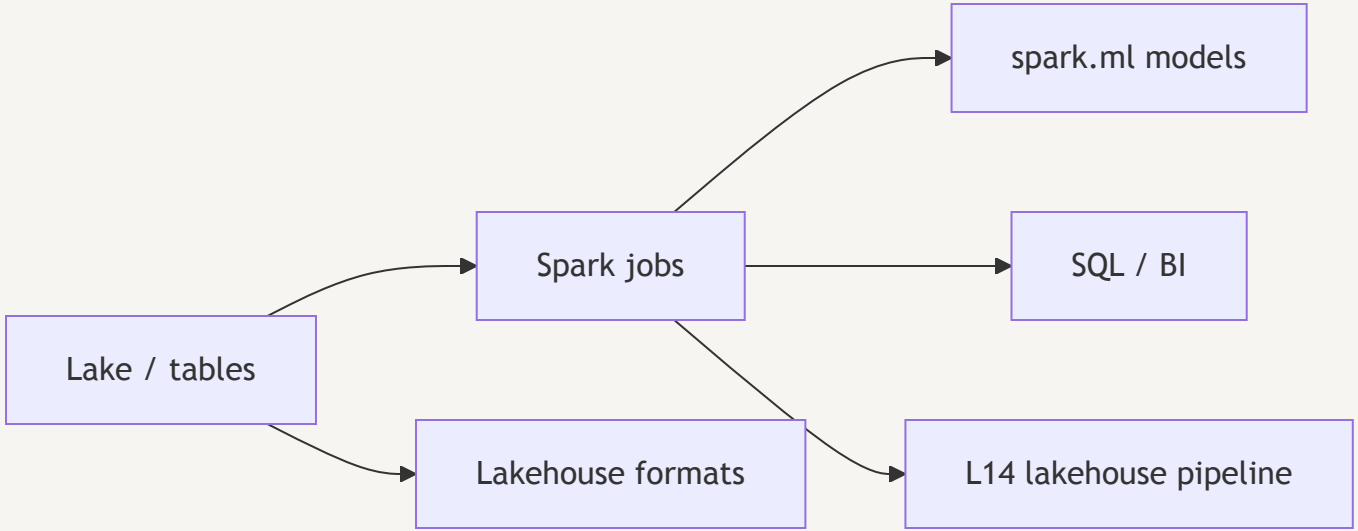
Demo purpose & steps

Purpose: End-to-end `spark.ml` on a sample dataset (lecture demo / lab notebook).

Step	Action
1	Load CSV/Parquet → DataFrame; inspect schema
2	Train/test split
3	Build Pipeline (features + estimator)
4	Optional: `CrossValidator` / param grid
5	`fit(train)` → `transform(test)`
6	Evaluator metrics · show a few rows
7	Save PipelineModel

Lab walks a notebook; this slide is the checklist.

From analytics to the platform



ML is one consumer of curated tables — next: **L14** end-to-end lakehouse pipeline (Kafka · Iceberg · dbt · GE · Airflow).

Takeaways

1. Prefer DataFrame API `spark.ml` — RDD `mllib` is maintenance-only
2. **Estimator.fit** → **Transformer.transform**; **Pipeline** locks feature+model steps
3. Columns `features` / `label` / `prediction` are the contract
4. Tune with **ParamGrid + CrossValidator** (or TVS); watch cost & leakage
5. **Persist** PipelineModel for scoring; demo closes the loop in lab

Next

L14 — End-to-end lakehouse pipeline